SF2957 - project 1

Group 9: Tommaso Brambilla, Ramon Padilla, Matilde Reina, Valentina Tomassini

November 2025

1 Stochastic subgradients for training support vector machines

1.a) Convexity of $R^{\text{emp}}(\Theta)$

In order to prove the convexity of $R^{\text{emp}}(\Theta)$ we are going to use the following basic properties about convexity:

- The sum of convex functions is convex;
- The maximum of a family of convex functions is convex.

This said, we can prove that

$$R^{\text{emp}}(\Theta) = \frac{1}{N}\sum_{k=1}^{n} \max_{j \neq y} \left[1 + \sum_{i=1}^{d} x_i(\Theta_{ij} - \Theta_{iy})\right]_+$$

is convex by only proving that functions of the form

$$f_j(\Theta) = 1 + (\Theta_j - \Theta_y)^T x$$

are convex. In order to prove it, we will use the following definition of convex function:
A function f is convex iff

$$f(\lambda x + (1-\lambda)y) \leq \lambda f(x) + (1-\lambda)f(y) \quad \forall x, y, \forall \lambda \in [0,1]$$

Proof:

$$f(\lambda\theta_i + (1-\lambda)\theta_j) \leq \lambda f(\theta_i) + (1-\lambda)f(\theta_j)$$

$$\implies \quad 1 + (\lambda\theta_i + (1-\lambda)\theta_j - \theta_y)^T x \leq \lambda(1 + (\theta_i - \theta_y)^T x) + (1-\lambda)(1 + (\theta_j - \theta_y)^T x)$$

Let us analyze the right-hand side term:

$$\lambda(1 + (\theta_i - \theta_y)^T x) + (1-\lambda)(1 + (\theta_j - \theta_y)^T x) =$$

$$= \lambda + \lambda(\theta_i - \theta_y)^T x + 1 + (\theta_j - \theta_y)^T x - \lambda - \lambda(\theta_j - \theta_y)^T x$$

$$= 1 + \lambda(\theta_i - \theta_y)^T x + (1-\lambda)(\theta_j - \theta_y)^T x$$

$$= 1 + (\lambda\theta_i + (1-\lambda)\theta_j - \theta_y)^T x$$

We just proved that the right-hand side term is equal to the left-hand side term, meaning that the relation $\leq$ is holding as an equality. Thus, the functions f_j are convex, implying that $R^{\text{emp}}(\Theta)$ is convex.

1.b) Subgradient of $R^{\text{emp}}(\Theta)$

Given the convexity properties of $R^{\text{emp}}(\Theta)$ and $L(\Theta, (x_i, y_i))$ as described in subsection 1.a), for any choice of per-sample subgradients G_i of $L(\Theta, (x, y))$ at Θ,

$$G \;=\; \frac{1}{m}\sum_{i=1}^{m} G_i \quad \in \quad \partial_\Theta R^{\text{emp}}(\Theta).$$

For a fixed sample (x_i, y_i) we define:

$$s_{i,j}(\Theta) = 1 + \theta_j^\top x_i - \theta_{y_i}^\top x_i, \qquad j \neq y_i,$$

$$L_i(\Theta) = \max\Big(0, \max_{j \neq y_i} s_{i,j}(\Theta)\Big).$$

Let the set of maximizers be

$$J_i = \big\{ j \neq y_i : \; s_{i,j}(\Theta) = \max_{k \neq y_i} s_{i,k}(\Theta) \big\}.$$

If $L_i(\Theta) = 0$, which implies all scores are non-positive, the constant 0 is the active piece. Thus the zero matrix is a valid subgradient:

$$G_i = 0 \in \partial_\Theta L(\Theta, (x_i, y_i)).$$

If $L_i(\Theta) > 0$, then the active pieces at Θ are the functions indexed by J_i. For each $j \in J_i$ define the matrix $G_i^{(j)} \in \mathbb{R}^{d \times K}$ by columns

$$G_{i,:,k}^{(j)} = \begin{cases} x_i, & k = j, \\ -x_i, & k = y_i, \\ 0, & \text{otherwise.} \end{cases}$$

The subgradient of $L(\Theta, (x_i, y_i))$ is

$$\partial_\Theta L(\Theta, (x_i, y_i)) = \operatorname{conv}\big\{ G_i^{(j)} : j \in J_i \big\}.$$

Therefore, any subgradient for sample i is a convex combination

$$G_i = \sum_{j \in J_i} \alpha_{i,j} G_i^{(j)}, \qquad \alpha_{i,j} \geq 0, \quad \sum_{j \in J_i} \alpha_{i,j} = 1.$$

And, consequently,

$$G = \frac{1}{m}\sum_{i=1}^{m} \begin{cases} 0, & \text{if } \max_{j \neq y_i} s_{i,j}(\Theta) \leq 0, \\ \displaystyle\sum_{j \in J_i} \alpha_{i,j}\, G_i^{(j)}, & \text{if } \max_{j \neq y_i} s_{i,j}(\Theta) > 0, \end{cases}$$

where each $G_i^{(j)}$ has x_i in column j, $-x_i$ in column y_i, and zeros elsewhere.

1.c) Implementation of subgradient and gradient descent

The code is available in Appendix.

We have chosen to implement the subgradient such that $\alpha_{i,j} = 1, \alpha_{i,k \neq j} = 0$ for a random $j \in J_i$. It makes the implementation simple and efficient.

1.d) Model performance w.r.t. training data size

The model performs very poorly initially but shows a rapid improvement as the training samples increase, yielding under 20% error rate with 100 samples. Limited data regimes suffer from high variance in the fitted model, and consequently in performance on the test set.

Performance starts to converge at around 500 samples, after which marginal performance improvement per increase in data size becomes small. This results seem to indicate that the model is able to capture the majority of the useful features within the first few hundred samples.

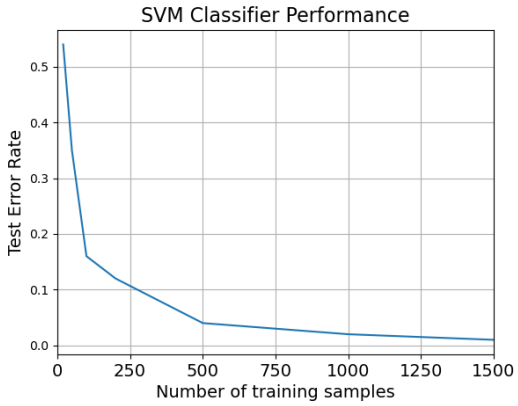


Figure 1: Model Error rate w.r.t. number of training samples. 40,000 sub-gradient iterations in training

1.e) Model performance w.r.t. learning rate

We analyze the influence of the learning rate on the classifier's performance, by varying $\alpha \in \{0.1, 0.25, 0.5, 0.75, 1.0\}$ for a LR $\epsilon_k \propto k^{-\alpha}$. As shown in Figure 2, for 400 iterations performance still has a component of randomness, as iterations increase and the model performance converges, medium small learning rates, particularly $\alpha \in \{0.25, 0.5\}$, consistently achieve limited but significantly better performance, yielding the lowest test error rates, with $\alpha = 0.5$ in particular achieving the best performance in limited data regimes. In contrast, the most extreme parameters investigated ($\alpha \in \{0.1, 1.0\}$) results in significantly higher error rates. On the one hand, too small a step size might fail to converge in the limited iterations, we can observe significant improvement as iterations increase. On the other hand, too aggressive a step size might struggle to converge to the minimum.

Thus, it seems a learning rate that decays too rapidly prevents the model from effectively reaching the global minimum, while a slower decay allows the classifier to approximate the minimum over as the iterations increase. The original $\epsilon_k \propto k^{-0.5}$ yields the best results across the different settings.

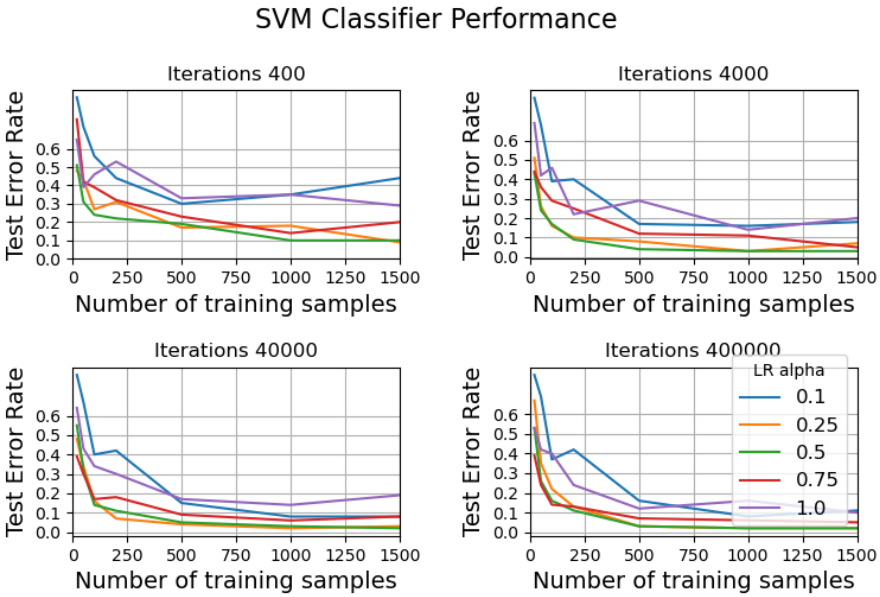


Figure 2: Model Error rate w.r.t. learning rate. Evaluated at different iteration ranges

2 Gaussian process classification

In the second part of the project, we want to investigate the performance of Gaussian processes (GP) classification and regression.

2.a) Gaussian Process Classification (GPC)

Gaussian Process Classification (GPC) is a nonparametric Bayesian approach for supervised classification that places a prior directly over functions, allowing flexible, probabilistic modeling. The defining characteristic of a GP is its kernel (covariance function), which expresses prior beliefs about the regularity and structure of the target function.
In this experiment, we classify handwritten digits ($8*8$ grayscale images flattened to 64 features) using GPC with various kernels and compare their performances.
We compared performances of Gaussian classifiers using four different kind of kernels from scikit-learn [2]:

- **Radial Basis Function:** $k_{\text{RBF}}(x, x') = \sigma^2 \exp\left(-\frac{2\|x-x'\|^2}{\ell^2}\right)$
  "Nearby" points are highly correlated; as distance grows, the covariance drops off very quickly. This kernel assumes the underlying function is very smooth
- **DotProduct:** $k_{\text{DotProduct}}(x, x') = \sigma_0^2 + x \cdot x'$
  Linear kernel (+ bias). This kernel represents the assumption that the function is linear in the input space (possibly with an offset)
- **Matern:** $k_{\text{Matern}}(x, x') = \sigma^2 \left(1 + \ell\|x - x'\|\right) \exp(-\ell\|x - x'\|)$
  Produces less-smooth functions than RBF, with controlled roughness that might match less-smooth or more "piecewise" patterns
- **Rational Quadratic:** $k_{\text{RQ}}(x, x') = \sigma^2 \left(1 + \frac{\|x-x'\|^2}{2\alpha\ell^2}\right)^{-\alpha}$
  handles functions with varying degrees of smoothness

Notice that in our implementation, the values of the hyper-parameters of all the four kernels (e.g. length-scale for RBF and bias for DotProduct) were optimized by maximizing the log-marginal-likelihood based on the passed optimizer [2].

Results for each tested kernel:

- **RBF:** Extremely poor performance (test error $\approx 90\%$ for all training sample sizes)
  The RBF kernel, with small lengthscale, struggles in high-dimensional, sparse image spaces. It fails to generalize. Many points look "far apart," so the prior covariance matrix is nearly diagonal ("no sharing information across points"), leading to overfitting: perfect on train, random on test.
- **DotProduct:** Very strong; (low test error, it goes from $\approx 40\%$ with training sample size 20 to $\approx 5\%$ with training sample size 1500)
  The DotProduct kernel acts like a linear classifier in feature space and high-dimensional data often favors linear methods unless specific local structure exists.
- **Matern:** Worse than DotProduct, better than RBF (test error goes from $\approx 80\%$ with training sample size 20 to $\approx 40\%$ with training sample size 1500)
  The roughness allows fitting less-smooth, more varied patterns than RBF, but it still suffers from the "curse of dimensionality" because distances between points are large, so the kernel can't generalize well between different images.
- **RQ:** Excellent; lowest test errors of all kernels (test error decreases a lot, going from $\approx 90\%$ with training sample size 20 to $\approx 4\%$ with training sample size 1500)
  This kernel is "multi-scale": models functions with both long-range trends and fine-grained local variations. In high-dimensional data like MNIST, this flexibility helps to capture patterns across various scales (some digits may be broad strokes, others fine details).

In the table and plot below, we show the errors obtained using GPC with different kernels, using a test set of 100 samples, and increasing the size of the training set (from 20 to 1500), as in part 1.

Kernel	**20**	**50**	**100**	**200**	**500**	**1000**	**1500**
RBF	0.87	0.83	0.86	0.90	0.89	0.92	0.87
DotProduct	0.36	0.23	0.11	0.05	0.08	0.04	0.05
Matern	0.80	0.70	0.67	0.56	0.44	0.54	0.42
RationalQuadratic	0.87	0.67	0.31	0.12	0.07	0.04	0.04

Table 1: Test error rates for different kernels across varying training sample sizes for GPC.

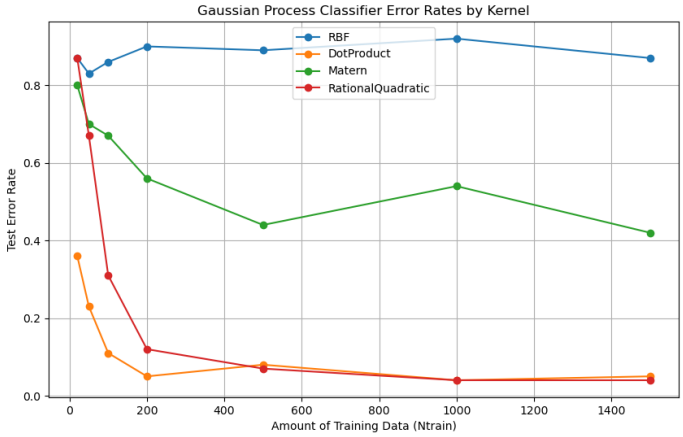


Figure 3: GPC test error rates for different kernels across varying training sample sizes

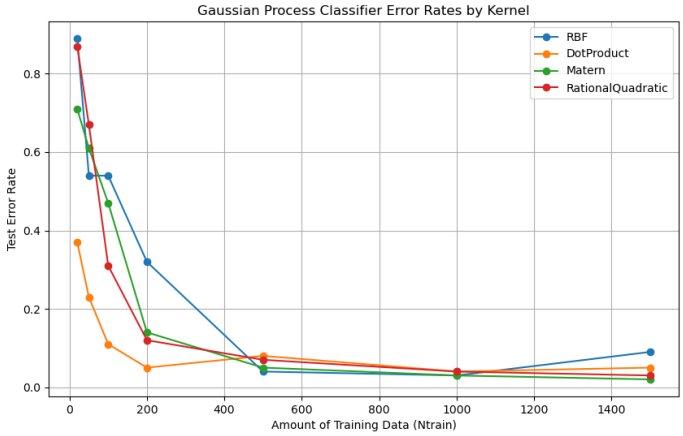


Figure 4: GPC test error rates using n restarts optimizer

n restarts optimizer : indicates the number of restarts of the optimizer function into the GP Classifier implementation, finding the kernel's parameters which maximize the log marginal likelihood. [2]

2.b) Gaussian Process Regression (GPR)

After investigating Gaussian process classification, we want to do the same with Gaussian process regression. In this approach, multi-class classification is performed by training one Gaussian Process regressor for each class $k \in \{0, \ldots, 9\}$. The targets are transformed into one-hot vectors, so that for class k the training labels are

$$y_i^{(k)} = \begin{cases} 1 & \text{if } y_i = k \\ 0 & \text{otherwise.} \end{cases}$$

For each class-specific regressor the model is

$$y_i^{(k)} = f_k(x_i) + \epsilon_i, \qquad \epsilon_i \sim \mathcal{N}(0, \sigma^2),$$

which corresponds to the standard Gaussian likelihood used in regression. Given the training inputs X, the kernel matrix K and the one-hot vector $Y^{(k)}$, the posterior mean used for prediction is

$$m_k(x_*) = k(x_*, X)\,(K + \sigma^2 I)^{-1} Y^{(k)}.$$

The matrix $(K+\sigma^2 I)$ is invertible for any $\sigma^2 > 0$ and remains well-conditioned in practice, which ensures that the posterior mean is stable under numerical inversion. Since the Gaussian likelihood leads to a closed-form Gaussian posterior, the regression formulation avoids approximation steps and the corresponding sensitivity to the kernel choice.

For a new input x_*, all ten posterior means $m_k(x_*)$ are computed, and the predicted class is obtained by the simple rule

$$\hat{y}(x_*) = \arg\max_k \; m_k(x_*).$$

Each regressor therefore estimates a smooth function that takes values close to 1 in regions associated with class k and close to 0 elsewhere. Smooth kernels such as the RBF, Matern and Rational Quadratic perform particularly well because they naturally capture the kind of smoothness needed to approximate one-hot functions. The DotProduct kernel, being linear, performs worse because a linear regressor cannot accurately approximate the non-linear one-hot indicator functions required for classification.

Overall, Gaussian Process Regression with one-hot encoding yields a simple and robust multi-class classifier. The closed-form posterior and the smoothness constraints imposed by the kernel allow the method to perform consistently well across different kernels and training set sizes.

In the table and plot below, we show the errors obtained using GPR with different kernels, using a test set of 100 samples, and increasing the size of the training set (from 20 to 1500).

Kernel	**20**	**50**	**100**	**200**	**500**	**1000**	**1500**
RBF	0.68	0.37	0.10	0.04	0.02	0.02	0.02
DotProduct	0.44	0.68	0.23	0.13	0.08	0.04	0.04
Matern	0.48	0.33	0.10	0.04	0.02	0.02	0.02
RationalQuadratic	0.40	0.33	0.10	0.04	0.02	0.02	0.02

Table 2: Test error rates for different kernels across varying training sample sizes for GPR.

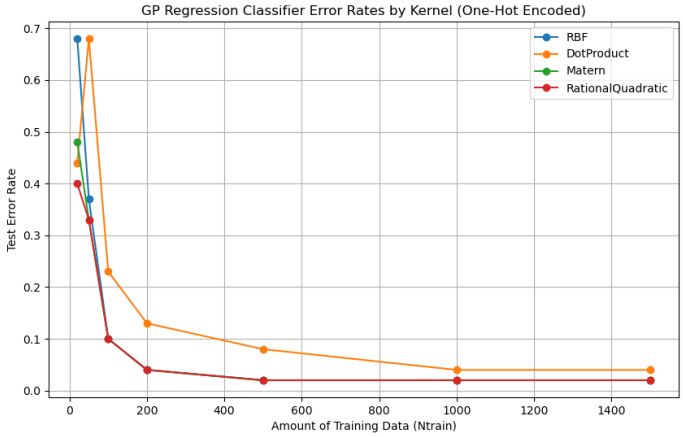


Figure 5: GPR test error rates for different kernels across varying training sample sizes

3 Convolutional neural network classification

3.a) Simple CNN

We build a CNN comprising two convolutional layers for feature extraction, followed by two fully connected layers for classification. The activation function employed was ReLU. The architecture choices are based on the tutorial from torch [1]. The code is available in Appendix.

Using the same test set of 100 samples , 80% of the remaining samples as training data (1278) and the rest as validation (319), the model achieves 0.00 error rate after training for 100 epochs.

3.b) Performance w.r.t parameters and training data size

To find the smallest network without significant performance degradation, we have iterated over several configurations. Initially we attempted to remove some of the layers, to find out that performance was not impacted by removing one of the convolutional layers as well as one of the linear layers.

Additionally we use max pooling to reduce the size of the convolutional layer output, and consequently reduce the parameters of the linear layer. With max pooling the largest value from a defined window is chosen, discarding the remaining values [1]. We have performed an iterative search of the best configuration of number of output channels for the convolutional layer Figure 6. The results show that the CNN achieves good performance. In particular, it is shown that a small model of 6 output channels, and 610 parameters, representing a 96.68% reduction with respect to the original model(18,346 parameters), is still capable of achieving 0.00 error rate. We can observe that performance converges around 0.01-0.00 for bigger models. Additionally, 0.01 error rate is our precision in measuring performance, therefore it is possible that a smaller model with 4 or 5 output channels would not sacrifice much performance.

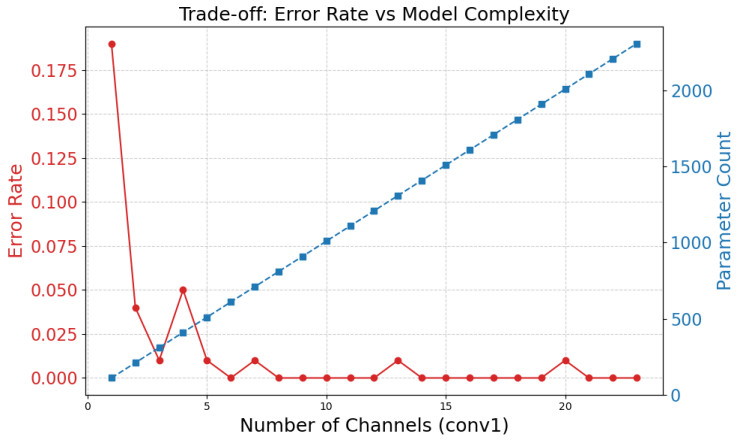


Figure 6: Trade off: error rate vs complexity in CNN

Moreover, we have investigated the performance with respect to the training size. Results show, Figure 7, that while using all the data very good performance can be achieved with small models. However, model size does significantly impact performance in limited data regimes, where our initial larger model consistently outperforms the smaller models.

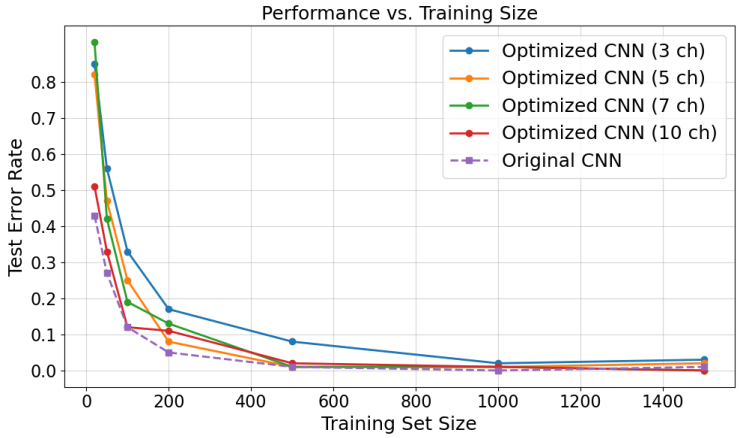


Figure 7: CNN Error rate w.r.t. training set size. Comparison between original and optimized architectures

4 Discussion: Model comparison

In this project, we implemented three different types of classifiers on the handwritten digits dataset: a linear SVM trained using the stochastic gradient descent, Gaussian processes classifier and regressor and CNNs.

The SVM is the simplest model that we implemented. Despite its simplicity, the model seems to work quite well once enough data is provided. In particular, in Figure 1 we saw that, with a limited set of 500 samples, the performance starts to converge, achieving an error rate of 5% and eventually converging to 1%.

Gaussian processes, depending on whether we use classification or regression, the performance changes dramatically. First of all, we see that GPC is highly sensitive to the choice of kernel. The model performs extremely poorly with RBF kernel, while it works well with DotProduct and Rational Quadratic kernels. Moreover, we also saw that it was by far the slowest method to train, which means that its applicability is limited. On the other hand, GPR outperforms GPC, achieving lower error rates even for a smaller size of

training data samples, and is not sensitive to the choice of kernel. Moreover, for small training sizes and particular kernels, Figure 5, GPR is the best performing method, surpassing both the SVM and the CNN in that regime, achieving 4% error with training sets limited to 200 samples.

Finally, CNN outperforms all the methods discussed above when the size of the training sample is big enough. Even with a simple architecture, we can achieve an error rate of $\approx 1-2\%$, and as we increase model complexity 0% test error is quickly achieved.

References

[1] PyTorch developers. *Learn the Basics*. Part of the "Deep Learning with PyTorch: A 60 Minute Blitz" tutorial series. PyTorch. URL: `https://docs.pytorch.org/tutorials/beginner/basics/intro.html` (visited on 11/25/2025).

[2] scikit-learn developers. *Gaussian Processes*. scikit-learn stable documentation. scikit-learn. URL: `https://scikit-learn.org/stable/modules/gaussian_process.html`.

Appendix

Stochastic Subgradients

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.utils import shuffle
import random

# import some data to play with
digits = datasets.load_digits()


#choose a seed
seed = 0
np.random.seed(seed)
# Load data into train set and test set
digits = datasets.load_digits()
X = digits.data
y = np.array(digits.target, dtype = int)
X,y = shuffle(X,y)
N,d = X.shape
Ntest = 100
Xtrain = X[0:-Ntest,:]
ytrain = y[0:-Ntest]
Xtest = X[-Ntest:,:]
ytest = y[-Ntest:]

def svmsubgradient(Theta, x, y):
    """
    Returns a subgradient of the objective empirical hinge loss.

    The inputs are Theta, of size n-by-K, where K is the number of classes,
    x of size n, and y an integer in {0, 1, ..., 9}.
    """
    G = np.zeros(Theta.shape)
    # Compute loss for all j
    scores = Theta.T.dot(x)
    margins = 1 + scores - scores[y]

    margins[y] = -np.inf # So iter's never picked as max

    max_loss = np.max(margins)
    if max_loss > 0: # check condition of gradient
        max_idx = np.where(margins == max_loss)[0]
        max_j = np.random.choice(max_idx)

        # Update
        G[:, max_j] += x
        G[:, y] -= x
    return(G)

def sgd(Xtrain, ytrain, maxiter = 10, init_stepsize = 1.0, l2_radius = 10000, alpha=0.5):
    """
    Performs maxiter iterations of projected stochastic gradient descent
    on the data contained in the matrix Xtrain, of size n-by-d, where n
    is the sample size and d is the dimension, and the label vector
    ytrain of integers in {0, 1, ..., 9}. Returns two d-by-10
    classification matrices Theta and mean_Theta, where the first is the final
    point of SGD and the second is the mean of all the iterates of SGD.

    Each iteration consists of choosing a random index from n and the
    associated data point in X, taking a subgradient step for the
    multiclass SVM objective, and projecting onto the Euclidean ball
    The stepsize is init_stepsize / sqrt(iteration).
    """
    K = 10
```

```
    NN, dd = Xtrain.shape
    # print(NN)
    Theta = np.zeros(dd*K)
    Theta.shape = dd,K
    mean_Theta = np.zeros(dd*K)
    mean_Theta.shape = dd,K
    for iter in range(maxiter):
        i = random.randint(0, NN-1)
        x = Xtrain[i,:]
        y = ytrain[i]

        G = svmsubgradient(Theta, x, y)
        stepsize = init_stepsize / ((iter + 1)**alpha)

        Theta = Theta - stepsize * G

        # Projection onto l2 ball
        norm = np.linalg.norm(Theta)
        if norm > l2_radius:
            Theta *= l2_radius / norm
        # Update mean
        mean_Theta = (iter/(iter+1)) * mean_Theta + (1/(iter+1)) * Theta

    return Theta, mean_Theta
```

Gaussian Process Classification

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.utils import shuffle
from sklearn.gaussian_process import GaussianProcessClassifier
from sklearn.gaussian_process.kernels import RBF, DotProduct, Matern, RationalQuadratic
from sklearn.preprocessing import StandardScaler

seed = 0
np.random.seed(seed)

digits = datasets.load_digits()
X = digits.data
y = np.array(digits.target, dtype=int)
X, y = shuffle(X, y)
N, d = X.shape

kernels = [
    1.0 * RBF(length_scale=1.0),
    DotProduct(sigma_0=1.0),
    1.0 * Matern(length_scale=1.0, nu=1.5),
    RationalQuadratic(length_scale=1.0, alpha=1.0,
                      length_scale_bounds=(1e-3, 1e3),
                      alpha_bounds=(1e-3, 1e9))
]

train_sizes = [20, 50, 100, 200, 500, 1000, 1500]
test_errors = np.zeros((4, len(train_sizes)))

# Static test set
Ntest = 100
Xtest = X[-Ntest:, :]
ytest = y[-Ntest:]

for j, Ntrain in enumerate(train_sizes):
    Xtrain = X[0:Ntrain, :]
    ytrain = y[0:Ntrain]

    scaler = StandardScaler().fit(Xtrain)
    X_train_s = scaler.transform(Xtrain)
```

```
    X_test_s = scaler.transform(Xtest)
    print('Number of samples', N)

    for kern, kernel in enumerate(kernels):
        print("\nTesting kernel:", kernel)
        gpc = GaussianProcessClassifier(kernel=kernel).fit(Xtrain, ytrain)
        yp_train = gpc.predict(Xtrain)
        train_error_rate = np.mean(np.not_equal(yp_train, ytrain))
        yp_test = gpc.predict(Xtest)
        test_error_rate = np.mean(np.not_equal(yp_test, ytest))
        print('Train error rate', train_error_rate)
        print('Test error rate', test_error_rate)
        test_errors[kern, j] = test_error_rate

print(test_errors)
```

Gaussian Process Regression

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import datasets
from sklearn.utils import shuffle
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.gaussian_process import GaussianProcessRegressor
from sklearn.gaussian_process.kernels import RBF, DotProduct, Matern, RationalQuadratic,
                                             ExpSineSquared

seed = 0
np.random.seed(seed)

digits = datasets.load_digits()
X = digits.data
y = np.array(digits.target, dtype=int)
X, y = shuffle(X, y)
N, d = X.shape

train_sizes = [20, 50, 100, 200, 500, 1000, 1500]
test_errors = np.zeros((4, len(train_sizes)))

# Static test set
Ntest = 100
Xtest = X[-Ntest:, :]
ytest = y[-Ntest:]

for j, Ntrain in enumerate(train_sizes):
    Xtrain = X[0:Ntrain, :]
    ytrain = y[0:Ntrain]

    scaler = StandardScaler().fit(Xtrain)
    X_train_s = scaler.transform(Xtrain)
    X_test_s = scaler.transform(Xtest)

    kernels = {
        "RBF": 1.0 * RBF(length_scale=1.0),
        "DotProduct": DotProduct(sigma_0=1.0, sigma_0_bounds=(1e-5, 1e5)),
        "Matern": 1.0 * Matern(length_scale=1.0, nu=1.5),
        "RationalQuadratic": RationalQuadratic(length_scale=1.0, alpha=1.0,
                                               length_scale_bounds=(1e-3, 1e3),
                                               alpha_bounds=(1e-3, 1e9))
    }

    encoder = OneHotEncoder(categories=[range(10)], sparse_output=False, handle_unknown='
                                                    ignore')
    y_train_onehot = encoder.fit_transform(ytrain.reshape(-1, 1))
    print(y_train_onehot.shape)
    print('Number of samples', N)
```

```
    def GP_regression_classifier(kernel):
        regressors = []
        for k in range(10):
            gpr = GaussianProcessRegressor(kernel=kernel)
            gpr.fit(X_train_s, y_train_onehot[:, k])
            regressors.append(gpr)

        predictions = []
        for reg in regressors:
            preds = reg.predict(X_test_s)
            predictions.append(preds)
        preds_matrix = np.vstack(predictions).T

        y_pred = np.argmax(preds_matrix, axis=1)
        test_error = np.mean(y_pred != ytest)
        return test_error, preds_matrix

    kern = 0
    for name, kernel in kernels.items():
        print(f"Testing kernel: {name}")
        error, _ = GP_regression_classifier(kernel)
        test_errors[kern, j] = error
        kern += 1
        print(f"Test error rate = {error:.4f}\n")

print(test_errors)
```

Convolutional Neural Network

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import TensorDataset, DataLoader, random_split
from sklearn.utils import shuffle
from sklearn import datasets
import copy
import matplotlib.pyplot as plt

seed = 0
np.random.seed(seed)
torch.manual_seed(seed)

digits = datasets.load_digits()
X = digits.data
y = np.array(digits.target, dtype=int)
N, d = X.shape
X, y = shuffle(X, y, random_state=seed)

Ntrain = 1697
Ntest  = 100

X = X.reshape(N, 1, 8, 8)
y = y

Xtrain = X[:Ntrain]; ytrain = y[:Ntrain]
Xtest = X[-Ntest:]; ytest = y[-Ntest:]

# Torch tensor
Xtrain_t = torch.tensor(Xtrain, dtype=torch.float32)
ytrain_t = torch.tensor(ytrain, dtype=torch.long)
Xtest_t  = torch.tensor(Xtest, dtype=torch.float32)
ytest_t  = torch.tensor(ytest, dtype=torch.long)

# Torch dataset
train_full_set = TensorDataset(Xtrain_t, ytrain_t)
test_set       = TensorDataset(Xtest_t, ytest_t)
```

```
val_size = int(0.2 * len(train_full_set))
train_size = len(train_full_set) - val_size

training_set, val_set = random_split(
    train_full_set,
    [train_size, val_size],
    generator=torch.Generator().manual_seed(seed)
)

# Dataloaders
train_loader = DataLoader(training_set, batch_size=32, shuffle=True)
val_loader   = DataLoader(val_set, batch_size=32)
test_loader  = DataLoader(test_set, batch_size=32)

class SimpleCNN(nn.Module):
    def __init__(self):
        super().__init__()
        # Input: (1, 8, 8)
        self.conv1 = nn.Conv2d(1, 8, kernel_size=3)   # (8, 6, 6)
        self.conv2 = nn.Conv2d(8, 16, kernel_size=3)  # (16, 4, 4)
        self.fc1 = nn.Linear(16 * 4 * 4, 64)
        self.fc2 = nn.Linear(64, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.relu(self.conv2(x))
        x = x.view(-1, 16*4*4)      # flatten
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x

class OptimizedCNN(nn.Module):
    def __init__(self, num_channels=5):
        super().__init__()
        # Input: (1, 8, 8)
        self.conv1 = nn.Conv2d(1, num_channels, kernel_size=3) # (n_ch, 6, 6)
        # Pooling reduces to (n_ch, 3, 3)
        self.fc = nn.Linear(num_channels * 3 * 3, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = F.max_pool2d(x, 2)
        x = x.view(x.size(0), -1)
        x = self.fc(x)
        return x

def train_evaluate(model, train_loader):
    optimizer = torch.optim.Adam(model.parameters(), lr=0.002)
    loss_fn = nn.CrossEntropyLoss()

    epochs = 30
    best_loss = float('inf')
    best_wts = copy.deepcopy(model.state_dict())

    # Training Loop
    for epoch in range(epochs):
        model.train()
        for xb, yb in train_loader:
            optimizer.zero_grad()
            loss = loss_fn(model(xb), yb)
            loss.backward()
            optimizer.step()

        # Validation
        model.eval()
        val_loss = 0.0
        with torch.no_grad():
```

```
            for xb, yb in val_loader:
                val_loss += loss_fn(model(xb), yb).item()

        if val_loss < best_loss:
            best_loss = val_loss
            best_wts = copy.deepcopy(model.state_dict())

    # Best weights
    model.load_state_dict(best_wts)

    # Test Evaluation
    model.eval()
    correct = 0
    total = 0
    with torch.no_grad():
        for xb, yb in test_loader:
            preds = model(xb)
            correct += (preds.argmax(dim=1) == yb).sum().item()
            total += yb.size(0)

    return 1.0 - (correct / total) # Error Rate
```